Karma Connect

Token Reduction Playbook

Generated 2026-06-16 - how to lower AI cost per prospect

The largest savings are already in place. A typical prospect now costs roughly $0.05-0.12. What remains are smaller levers that trade against message quality or features - so they are judgement calls, not automatic wins.

Already optimized

- Message variations per engine: 10 -> 3 (cut generation output ~70%).
- Company web research: 3 searches -> 1 (cut the biggest variable input).
- Hybrid model: Haiku for parsing + research, Sonnet for analysis + message generation (~1/3 cost on research-heavy prospects).

Before vs after (typical prospect, billed at 2,000/month)

Stage	Tokens	Cost/prospect	Per month
Now (baseline)	~16,300	$0.095	$191
+ Code tweaks (cache brand-voice + drop AI Coach)	~15,900	$0.084	$169
+ Usage discipline (run ~2 engines, not 5)	~8,700	$0.045	$89

Full recommended set: ~47% fewer tokens, ~53% lower cost (about $100/month saved at 2,000).

Notes: "Typical" = analysis + 4 engines (connection, relationship, 1 meeting, follow-up). Caching lowers cost without lowering raw token count - cached input is billed at ~10%. The big drop comes from usage discipline (opening only the engines you need), not the code tweaks.

A. Quick code tweaks (small-medium saving, low risk)

Cache the brand-voice block

Saves: Input tokens on every generation call within a prospect run.
Trade-off: None visible - the ~830-token humanize instructions move into the cached system prompt. Safest option.

Remove the "AI Coach" note under each draft

Saves: Output tokens (the pricier kind) on every draft.
Trade-off: You lose the one-line rationale shown under each message in the UI.

Slim the AI profile sent into generation

Saves: Input tokens per generation call (send only hooks + pain points + persona, not the full profile).
Trade-off: Slightly less context could marginally reduce personalization.

Drop the analysis auto-retry

Saves: A rare double-charge when the analysis JSON fails to parse.
Trade-off: A parse hiccup shows an error instead of silently retrying. Tiny saving.

B. Model choices (bigger cost saving, quality trade-off)

Push more engines to Haiku

Saves: ~2/3 on whichever engines move (e.g. connection/relationship on Haiku, keep Sonnet for meeting/follow-up).
Trade-off: Lower-quality short messages.

All-Haiku

Saves: ~2/3 across the board.
Trade-off: Noticeable drop in outreach-copy quality - not advised for the messages your team sends.

Fewer variations (3 -> 2)

Saves: ~1/3 more of generation output.
Trade-off: Less choice per engine.

C. Usage habits (often the biggest real-world lever, zero build)

- Only open the engines you will use. Each engine (connection / relationship / each meeting length / follow-up) is a separate paid call. Generating all of them "just in case" is the #1 hidden cost - using 2 instead of 5 is ~60% less per prospect.
- Skip company web-research when you already have the info. It is the single most expensive call (web pages pulled into context).
- Avoid "Regenerate all" unless needed - each regenerate is a fresh paid call.
- Paste a focused LinkedIn profile, not giant pages - the parser reads up to ~24k characters.

Recommended order

- 1. Cache the brand-voice block (A) - free of downsides.
- 2. Enforce usage discipline (C) - moves the needle more than any code change.

- 3. Optionally remove the AI Coach note (A) if the team does not use it.
- 4. Only consider model downgrades (B) if you need a step-change and accept lower message quality.

Reality check: beyond caching + usage discipline, you are shaving cents per prospect against rising quality risk. At ~$190/mo for 2,000 typical prospects, usage discipline will save more than further code changes. Exact per-call usage is recorded in the app's usage table (GET /api/usage).